AI 應用原型 · 產品流程設計 · 整合驗證

CatchUp
遊戲直播 AI 視覺回顧

讓晚進直播的觀眾，透過 **!recap** 跟上前情。

張育瑄 | 需求與流程設計、AI 協作開發及實測驗收 | 2026.11

Devpost 已投稿作品 ↗　約 2:45 英文示範 ↗　GitHub 程式碼 ↗

OBS 遊戲截圖 → OpenAI 視覺分析 → 共享摘要快取 → Twitch 指令回覆

9 項	1 次回覆	0 次
自動測試於交付時通過	2 次實際指令，另 1 次冷卻抑制	聊天室查詢額外觸發 AI

01 問題與產品取捨

晚進遊戲直播的觀眾缺乏前情，重複詢問容易打斷主播。以遊戲畫面作為證據，不依賴主播持續說話；抽樣分析建立共享摘要，避免每位觀眾查詢都重讀整段影片。

觀眾以 **!recap / !回顧** 取得文字摘要；本機回顧頁提供時間戳、圖片及不確定性提示。

02 我的角色與 Codex 協助

我：提出情境，決定視覺優先、最小功能、成本控制及驗收標準；操作帳號授權與 OBS 設定，提供實測回饋，確認並完成投稿。

Codex：協助產生與修改程式、測試、文件，整合除錯與執行檢查，準備示範及投稿素材。本作品為 AI 協作開發。

03 實作與驗證

- Node.js 24 原生模組，無外部 npm 套件依賴。
- OBS WebSocket、OpenAI Responses API、Twitch OAuth / EventSub / Send Chat Message。
- 冷卻、重複訊息抑制、預算預留及用量紀錄。
- 真實 TapTapLoot 截圖 → AI → 真實聊天室回覆；以主播帳號在未開台頻道驗證。

該場一次分析估算 US$0.004137，非帳單核對值，也非每小時成本。

04 限制與下一步

- 已投稿 Codex x TwitchCon Hackathon，尚無得獎結果。
- 尚未驗證公開直播、第二觀眾帳號或長時間變動遊戲的準確度。
- 不分析音訊，抽樣可能漏掉瞬間事件。
- 回顧頁尚無公開託管；靜態匯出需手動更新。
- 尚無使用者留存或節省工時成效數據。

免金鑰檢視

全新下載原始碼，使用 Node.js 24+ 執行 `node src/server.mjs`，開啟 localhost:3000，點 Simulate one minute、Viewer recap、Send。離線素材與事件為腳本示範，不呼叫 AI，也不能用來證明 AI 準確率。真實串接需操作者自行設定金鑰與授權。

更新：2026-11-02 | 本作品展示需求拆解、AI 工具協作與驗收能力；不宣稱所有程式碼均由本人獨立撰寫。公開素材不含 API key、私人帳務或職涯規劃資料。